

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## OPTIMIZACIÓN DE BASES DE DATOS

En la clase de hoy aprenderemos los principios de la optimización de consultas en SQL y una vez que interpretemos sus fundamentos, aplicaremos planes de ejecución, con lo cual lograremos diagnosticar problemas de rendimiento en las consultas SQL y proponer mejoras aplicando técnicas básicas de optimización usando SQL Workbench.

**¿QUE SE DENOMINA OPTIMIZACIÓN DE CONSULTAS?**

La optimización de consultas es el proceso mediante el cual el motor de base de datos determina la forma más eficiente de ejecutar una sentencia SQL. Esto se basa en estadísticas de los datos, índices existentes y heurísticas internas del sistema gestor.

El proceso de optimización de consultas implica que el sistema gestor de bases de datos pueda reordenar operaciones como los JOIN, las condiciones en la cláusula WHERE o los agrupamientos con GROUP BY, con el objetivo de mejorar el rendimiento.

Un aspecto fundamental es el uso adecuado de índices, lo cual implica seleccionar cuidadosamente qué columnas conviene indexar, ya que esto puede reducir significativamente el tiempo de búsqueda.

Es importante también evitar el uso de funciones sobre columnas indexadas, dado que esto impide que el índice sea utilizado por el optimizador.

Finalmente, se recomienda evitar el uso de SELECT \* y, en su lugar, especificar únicamente las columnas necesarias, ya que seleccionar todos los campos puede generar una sobrecarga innecesaria en el procesamiento y en la transferencia de datos.

**OPTIMIZACIÓN DE CONSULTAS: PUNTOS CLAVE**

🔗 **Reordenamiento de operaciones:** El optimizador de consultas puede reorganizar cláusulas como JOIN, WHERE y GROUP BY para mejorar el rendimiento de ejecución.

🔗 **Uso de índices:** Elegir adecuadamente qué columnas indexar es fundamental para acelerar las búsquedas y reducir la carga del sistema.

🔗 **Evitar funciones sobre columnas indexadas:** Aplicar funciones a columnas que tienen índices (por ejemplo, UPPER (nombre)) impide que el índice sea utilizado, afectando el rendimiento.

🔗 **Seleccionar solo lo necesario:** Usar SELECT \* genera una carga innecesaria de datos. Es preferible especificar únicamente las columnas requeridas en la consulta para optimizar el uso de recursos.

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## LOS PLANES DE EJECUCIÓN Y MEJORA DE CONSULTAS EN SQL

Un plan de ejecución es una representación gráfica o textual del camino que seguirá la base de datos para ejecutar una consulta. Herramientas como EXPLAIN (en MySQL o PostgreSQL) permiten visualizar este plan.

## ¿CUALES SON LOS COMPONENTES MÁS COMUNES PARA ANALIZAR UN PLAN?

Cuando analizamos un plan de ejecución de una consulta SQL, aparecen varios datos importantes que nos ayudan a entender su rendimiento. Algunos de los componentes más comunes son:

**Tipo de acceso:** indica cómo se accede a los datos. Puede ser de forma secuencial (revisando fila por fila), por índice (usando un índice para ubicar los datos más rápido), o acceso único (cuando se accede a una única fila por clave primaria o índice único).

**Filas estimadas:** es la cantidad de registros que el sistema estima que deberá leer. Cuantas más filas, más recursos puede requerir la consulta.

**Costo estimado:** representa una medida del esfuerzo o trabajo que necesitará el sistema para ejecutar la consulta. Aunque no tiene una unidad fija, sirve para comparar diferentes formas de ejecutar una misma consulta y elegir la más eficiente.

## ¿QUE SÍ, Y QUE NO DEBEMOS HACER CUANDO ESCRIBIMOS CONSULTAS EN SQL?

Al momento de optimizar consultas en SQL, aplicar ciertas estrategias puede hacer una gran diferencia en el rendimiento. Algunas **buenas prácticas** incluyen:

**Crear índices adecuados:** cuando usamos condiciones en la cláusula WHERE, es útil tener índices sobre esas columnas. Esto permite que la base de datos encuentre los datos más rápido, sin tener que recorrer toda la tabla.

**Reescribir JOIN innecesarios:** si una consulta tiene muchas uniones (JOIN), es importante revisar si todas son necesarias. A veces, se pueden eliminar sin afectar el resultado, lo cual simplifica y agiliza la ejecución.

**Evitar subconsultas complejas:** en algunos casos, una subconsulta puede reemplazarse por un JOIN, lo que mejora la legibilidad y la eficiencia. Esto es especialmente útil si la subconsulta se ejecuta muchas veces dentro del proceso.

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## DIAGNÓSTICO Y OPTIMIZACIÓN DE UNA CONSULTA

Cuando una consulta en una base de datos se ejecuta lentamente, es fundamental seguir un proceso sistemático para identificar y corregir el problema.

Este enfoque metódico permite optimizar consultas de manera efectiva y mejorar el rendimiento general de la base de datos.

El diagnóstico y la optimización es un proceso técnico que sigue etapas específicas:

**Observar el comportamiento de la consulta:** el primer paso es notar cuándo y cómo una consulta se vuelve lenta. Esto puede incluir tiempos de respuesta altos o bloqueos al acceder a datos.

**Analizar el plan de ejecución:** con herramientas como EXPLAIN (en MySQL o PostgreSQL) o EXPLAIN ANALYZE, se puede ver el recorrido exacto que realiza el motor de base de datos para ejecutar la consulta. Esto revela detalles técnicos que ayudan a entender el origen del problema.

**Detectar cuellos de botella:** el análisis permite encontrar operaciones costosas, como los “*full table scan*”, donde se recorren todos los registros de una tabla en lugar de usar un índice. Estas operaciones suelen ser las responsables de las demoras.

**Aplicar mejoras:** según el diagnóstico, se pueden implementar soluciones como:

- ☺ **Crear índices** en columnas utilizadas en filtros o uniones.
- ☺ **Reescribir la consulta** para hacerla más eficiente (por ejemplo, simplificando condiciones o reorganizando JOINS).
- ☺ **Aplicar particionamiento** en tablas grandes, para dividir los datos y reducir el volumen de búsqueda.

## ¿CUÁLES SON LAS HERRAMIENTAS PARA OPTIMIZAR Y MEJORAR CONSULTAS?

Los administradores de bases de datos (también los desarrolladores) pueden tomar decisiones informadas al momento de optimizar consultas y mejorar el rendimiento de los sistemas utilizando (dependiendo del SGBD) algunos comandos útiles para el análisis de rendimiento de consultas como **EXPLAIN** o **EXPLAIN PLAN FOR, ANALYZE** y **SET STATISTICS TIME ON**.

Ahora ampliaremos un poco sobre cada uno de estos comandos, es importante que amplíes con la bibliografía de la cátedra y la documentación técnica de cada entorno.

**BASE DE DATOS II****CONTENIDOS Y ACTIVIDADES****EXPLAIN o EXPLAIN PLAN FOR**

Muestra el plan de ejecución estimado que usará la base de datos para resolver una consulta, permitiéndonos ver cómo la base recorrerá las tablas (por ejemplo, si usará índices o hará un escaneo completo), lo que ayuda a anticipar problemas de rendimiento.

Al comando lo utilizamos en PostgreSQL, MySQL, MariaDB, SQLite y otros sistemas similares, en Oracle hay una pequeña diferencia, ya que lo escribimos como EXPLAIN PLAN FOR.

**ANALYZE**

A diferencia de EXPLAIN o EXPLAIN PLAN FOR, ANALYZE solo muestra si lo que se estimaba realmente sucedió, y si hubo diferencias importantes, lo que puede indicar problemas como estadísticas desactualizadas o índices mal elegidos, para lograrlo ejecuta la consulta realmente y proporciona el plan de ejecución con estadísticas reales (tiempo, cantidad de filas leídas, etc.).

Este comando se utiliza especialmente en PostgreSQL, cuando se combina con el comando EXPLAIN: EXPLAIN ANALYZE, en MySQL solo existía el comando EXPLAIN, que mostraba un plan estimado, sin ejecutar la consulta, pero a partir de la versión 8.0.18 se agregó una nueva versión de EXPLAIN ANALYZE, que sí ejecuta la consulta y muestra algunas métricas reales, como el tiempo de cada operación del plan y la cantidad de filas examinadas.

Aunque el EXPLAIN ANALYZE de MySQL se parece al de PostgreSQL, no brinda tanta información detallada, ni tan precisa sobre el consumo de recursos internos como sí lo hace PostgreSQL, además, la forma de presentar los datos es distinta y un poco más limitada.

**SET STATISTICS TIME ON**

Este comando solo lo podemos usar en SQL Server y muestra el tiempo real de CPU en que tardan en ejecutarse las instrucciones de una consulta y el tiempo total que le tomó ejecutar cada instrucción, permitiendo medir cuánto demora exactamente cada parte de una consulta, lo cual resulta útil para comparar versiones diferentes de una consulta y elegir la más rápida.

**PASANDO EN LIMPIO**

| HERRAMIENTA                   | FUNCIÓN PRINCIPAL                                                          | SGBD                                       |
|-------------------------------|----------------------------------------------------------------------------|--------------------------------------------|
| <b>EXPLAIN</b>                | Muestra el plan de ejecución estimado <i>(sin ejecutar la consulta)</i> .  | PostgreSQL, MySQL, MariaDB, SQLite, Oracle |
| <b>EXPLAIN ANALYZE</b>        | Ejecuta la consulta y muestra el plan real con tiempos y filas procesadas. | PostgreSQL, MySQL (desde 8.0.18)           |
| <b>SET STATISTICS TIME ON</b> | Informa tiempo de CPU y tiempo total de ejecución de cada consulta.        | SQL Server                                 |

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## TRABAJAMOS

Ahora pondremos manos a la obra para práctica con los diferentes motores SQL e ir viendo cómo usando los diferentes comandos obtendremos información relevante que nos permitan tomar decisiones informadas para optimizar consultas y en definitiva mejorar el rendimiento de nuestras bases de datos.

Primeramente, veamos un ejemplo en MySQL con el comando **EXPLAIN**

Supongamos que tenemos en la base de datos *mi\_empresa* y se nos pide obtener una lista de clientes junto con el monto total de los pedidos cuyo total supere los \$100.000,00; para lograrlo podemos construir una consulta que recupere los datos de las tablas *clientes* y *pedidos*, pero antes de presionar el botón ejecutar podemos ver como impactaría en el rendimiento general de nuestra base de datos su ejecución, para eso podemos previamente analizar la consulta con un plan de ejecución utilizando el comando **EXPLAIN** y ver que informe nos brinda.

## VAYAMOS AL SQL

CREAMOS UNA BASE DE DATOS *mi\_empresa* Y DOS TABLAS: *clientes* y *pedidos*

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100)  
);
```

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    cliente_id INT,  
    total DECIMAL(10,2),  
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

## CREAMOS LA CONSULTA

```
SELECT c.nombre, p.total  
FROM clientes c  
JOIN pedidos p ON c.id = p.cliente_id  
WHERE p.total > 100000;
```

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## PREVIAMENTE ANALIZAMOS SU PLAN DE EJECUCIÓN

```
EXPLAIN SELECT c.nombre, p.total
```

```
FROM clientes c
```

```
JOIN pedidos p ON c.id = p.cliente_id
```

```
WHERE p.total > 100000;
```

## DEBERIAMOS PODER VER EL SIGUIENTE RESULTADO

| id | select_type | table | type   | possible_keys | key     | rows | Extra       |
|----|-------------|-------|--------|---------------|---------|------|-------------|
| 1  | SIMPLE      | p     | ALL    | cliente_id    | NULL    | 5000 | Using where |
| 1  | SIMPLE      | c     | eq_ref | PRIMARY       | PRIMARY | 1    |             |

Fíjense lo importante del informe previo de ejecución, ya que nos permite observar el costo de procesamiento de la secuencia paso a paso, por ejemplo:

**PASO UNO:** la tabla **p** (*pedidos*) se recorre completa (*type = ALL*) porque no hay índice útil para **total**.

**PASO DOS:** no hay inconvenientes en acceder a la a tabla **c** (*clientes*) por su clave primaria (*eq\_ref*).

**PASO TRES:** el motor de base de datos primero filtra los pedidos cuyo total sea mayor a 100000 y luego hace un JOIN con clientes.

## ¿PODEMOS MEJORAR LA CONSULTA?

Podríamos agregar un **índice** sobre **total** y ver si mejoramos el rendimiento:

```
CREATE INDEX idx_total ON pedidos(total);
```

Después de crear el índice, si volvemos a ejecutar el comando **EXPLAIN**, el tipo de acceso a la tabla **p** (*pedidos*) podría pasar de **ALL** a **range**, lo cual mejora la eficiencia.

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## ES MOMENTO CONTINUAR TRABAJANDO

ANÁLISIS DE UNA CONSULTA CON **EXPLAIN**

Veremos el uso de la instrucción **EXPLAIN** para **analizar el plan de ejecución** de una consulta SQL y **mejorar su rendimiento**, para ello en primer lugar vamos a preparar entorno creando la base de datos **ventas**:

PRIMERO CREAMOS LA BASE DE DATOS **VENTAS**

```
CREATE DATABASE ventas_db;
```

```
USE ventas_db;
```

DESPUÉS CREAMOS LAS TABLAS **CLIENTES Y PEDIDOS**:

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100)  
);
```

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    cliente_id INT,  
    total DECIMAL(10,2),  
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

## INSERTAR DATOS DE PRUEBA

```
INSERT INTO clientes VALUES (1, 'Ana'), (2, 'Luis'), (3, 'Sofía');
```

```
INSERT INTO pedidos VALUES  
(101, 1, 150.00),  
(102, 1, 80.00),  
(103, 2, 200.00),  
(104, 3, 90.00);
```

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## PRIMERA PRÁCTICA SUPERVISADA CON EL COMANDO EXPLAIN

## ANÁLISIS DE LA CONSULTA

En primer lugar, vamos a trabajar con una consulta que nos devuelve una lista de clientes junto con los montos de sus pedidos mayores a 100.000, donde solo se incluirán aquellos clientes que tengan al menos un pedido con ese valor o más.

Para lograrlo se toma la tabla clientes y se le asigna el alias c para simplificar la escritura. (FROM clientes c)

Luego se realiza una unión interna (INNER JOIN) entre la tabla clientes y la tabla pedidos, la unión se establece donde el campo id de la tabla clientes coincide con el campo cliente\_id de la tabla pedidos, esto permite relacionar cada pedido con el cliente correspondiente. (JOIN pedidos p ON c.id = p.cliente\_id)

Seguidamente, se filtran los resultados para mostrar solo aquellos pedidos cuyo campo total sea mayor a 100.000. (WHERE p.total > 100000)

Finalmente, (SELECT c.nombre, p.total) se seleccionan y muestran dos columnas: **c.nombre** que representa el nombre del cliente y **p.total** el monto total del pedido

## ESCRIBIMOS

**SELECT** c.nombre, p.total

FROM clientes c

JOIN pedidos p ON c.id = p.cliente\_id

WHERE p.total > 100000;

## EL RESULTADO DE LA CONSULTA

| clientes |            |              |
|----------|------------|--------------|
| id       |            | nombre       |
| 1        |            | Juan Pérez   |
| 2        |            | Ana Gómez    |
| 3        |            | Carla Méndez |
| 4        |            | Lucas Sosa   |
| pedidos  |            |              |
| id       | cliente_id | total        |
| 101      | 1          | 150000       |
| 102      | 2          | 85000        |
| 103      | 3          | 110000       |
| 104      | 1          | 95000        |
| 105      | 4          | 120000       |



## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

APLICANDO **EXPLAIN**

**EXPLAIN SELECT** c.nombre, p.total

FROM clientes c

JOIN pedidos p ON c.id = p.cliente\_id

WHERE p.total > 100000;

## EL RESULTADO DEL EXPLAIN EJECUTADO

| id | select_type | table | type   | possible_keys | key     | key_len | ref          | rows | filtered | Extra       |
|----|-------------|-------|--------|---------------|---------|---------|--------------|------|----------|-------------|
| 1  | SIMPLE      | p     | ALL    | cliente_id    | NULL    | NULL    | NULL         | 5000 | 10.00    | Using where |
| 1  | SIMPLE      | c     | eq_ref | PRIMARY       | PRIMARY | 4       | p.cliente_id | 1    | 100.00   |             |

## INTERPRETEMOS LA TABLA DE RESULTADOS

**id:** es el número del identificador único asignado a cada tupla del informe.

**table:** informa el orden y nombre de la tabla que se analiza, en nuestro ejemplo primero la tabla pedidos (p) y luego la tabla clientes (c).

**type:** indica el tipo de acceso (e.g., ALL, index, ref), *ALL* significa *full scan* (no óptimo), *eq\_ref* indica una búsqueda eficiente por clave primaria.

**possible\_keys:** son las claves que podrían usarse para optimizar.

**key:** es la clave que realmente se usó o si se usó algún índice.

**key\_len:** indica la longitud del índice que el optimizador decidió usar, en bytes, nos dice cuántos bytes del índice realmente se están usando para ejecutar la consulta.

**ref:** indica qué columna o constante se está usando para comparar con el índice en la tabla actual. Muestra de dónde viene el valor que se está utilizando para hacer la búsqueda en el índice, se refiere a la columna de otra tabla (o constante) que está participando en el JOIN o en la condición del WHERE.

**rows:** es la cantidad estimada de filas a recorrer.

**filtered:** se trata del porcentaje estimado de filas que pasan el filtro (WHERE).

**Extra:** son notas adicionales; por ejemplo, si se aplica un filtro con WHERE.

## ¡ATENCIÓN!

Si aparece un "Using where" o "Using join buffer" en el campo **Extra**, puede haber oportunidad de optimización.

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## CREAR UN ÍNDICE SOBRE LA COLUMNA TOTAL EN LA TABLA PEDIDOS

```
CREATE INDEX idx_total ON pedidos(total);
```

## VOLVER A EJECUTAR LA CONSULTA CON EXPLAIN Y COMPARAR

```
EXPLAIN SELECT c.nombre, p.total
```

```
FROM clientes c
```

```
JOIN pedidos p ON c.id = p.cliente_id
```

```
WHERE p.total > 100000;
```

## EL RESULTADO DEL EXPLAIN EJECUTADO

| i<br>d | select_<br>type | tabl<br>e | typ<br>e | possible_keys     | key        | key_<br>length | ref     | row<br>s | Extra       |
|--------|-----------------|-----------|----------|-------------------|------------|----------------|---------|----------|-------------|
| 1      | SIMPLE          | c         | ALL      | PRIMARY           | NULL       | NULL           | NULL    | 4        |             |
| 1      | SIMPLE          | p         | ref      | cliente_id, total | cliente_id | 4              | db.c.id | 3        | Using where |

## OBSERVÁ EL RESULTADO Y RESPONDÉ

- ? ¿Se utilizó el índice?
- ? ¿Qué diferencias ves en la columna **type** y **rows**?
- ? ¿Qué indica esto sobre el impacto del índice?

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## RESPUESTAS DE LAS PREGUNTAS DE LA PRÁCTICA SUPERVISADA Y GUIADA

## PARA ESTA ACTIVIDAD UTILIZAMOS LA SIGUIENTE CONSULTA

**EXPLAIN SELECT** c.nombre, p.total

FROM clientes c

JOIN pedidos p ON c.id = p.cliente\_id

WHERE p.total > 100;

## Y EL RESULTADO FUE

| i<br>d | select_ty<br>pe | tabl<br>e | typ<br>e | possible_key<br>s    | key            | key_le<br>n | ref         | row<br>s | Extra              |
|--------|-----------------|-----------|----------|----------------------|----------------|-------------|-------------|----------|--------------------|
| 1      | SIMPLE          | c         | ALL      | PRIMARY              | NULL           | NULL        | NULL        | 4        |                    |
| 1      | SIMPLE          | p         | ref      | cliente_id,tot<br>al | cliente_i<br>d | 4           | db.c.i<br>d | 3        | Using<br>wher<br>e |

## ¿SE UTILIZÓ EL ÍNDICE?

Sí, **se utilizó el índice**. La columna key en la segunda fila muestra que se utilizó el índice cliente\_id en la tabla *pedidos*. Esto indica que se está usando un acceso optimizado para hacer el JOIN con la tabla clientes.

## ¿QUÉ DIFERENCIAS VES EN LA COLUMNA TYPE Y ROWS?

En la tabla *clientes*, el type es ALL, lo que indica un recorrido completo de la tabla (full table scan).

En la tabla *pedidos*, el type es ref, lo que indica un acceso por índice, más eficiente.

## ADEMÁS

rows en *clientes* = 4 (se estiman 4 filas a escanear completamente).

rows en *pedidos* = 3 (se estiman 3 filas a buscar mediante índice).

## ¿QUÉ INDICA ESTO SOBRE EL IMPACTO DEL ÍNDICE?

El uso del índice en la tabla pedidos reduce la cantidad de filas que necesita analizar el motor de base de datos y permite un acceso más eficiente. En cambio, como la tabla clientes no utiliza índice (type = ALL), su rendimiento podría verse afectado si tuviera muchas filas. Esto demuestra que el índice mejora la eficiencia al reducir el número de filas a leer y permite un acceso más directo a los datos.

## BASE DE DATOS II

### CONTENIDOS Y ACTIVIDADES

#### PRACTICA INTENSIVA

Ahora vamos a realizar una práctica intensiva, **seguí los pasos y andá tomando nota de todo lo que observás**, de esta manera lograrás comprender cómo interpretar un plan de ejecución utilizando la instrucción **EXPLAIN** en SQL, y cómo los índices afectan el rendimiento de una consulta.

#### ACTIVIDAD INTENSIVA 01

##### CONSULTA DE ANÁLISIS

EXPLAIN SELECT c.nombre, p.total

FROM clientes c

JOIN pedidos p ON c.id = p.cliente\_id

WHERE p.total > 100;

#### ESQUEMA PARA PRÁCTICA

| clientes |               |
|----------|---------------|
| id       | nombre        |
| 1        | Juan Pérez    |
| 2        | Ana Torres    |
| 3        | Luis Gómez    |
| 4        | Marta Delgado |

| pedidos |            |       |
|---------|------------|-------|
| id      | cliente_id | total |
| 1       | 1          | 50    |
| 2       | 1          | 120   |
| 3       | 2          | 300   |
| 4       | 4          | 90    |
| 5       | 3          | 500   |

#### PLAN DE EJECUCIÓN

Diseñá el plan de ejecución que te permita lograr ver el resultado como en la tabla que te comparto.

| id | select_type | table | type | possible_keys    | key        | key_len | ref     | rows | Extra       |
|----|-------------|-------|------|------------------|------------|---------|---------|------|-------------|
| 1  | SIMPLE      | c     | ALL  | PRIMARY          | NULL       | NULL    | NULL    | 4    |             |
| 1  | SIMPLE      | p     | ref  | cliente_id,total | cliente_id | 4       | db.c.id | 3    | Using where |

## BASE DE DATOS II

### CONTENIDOS Y ACTIVIDADES

#### RESPONDE LAS SIGUIENTES PREGUNTAS DE ANÁLISIS

- ❓ ¿Se utilizó algún índice en esta consulta? ¿En qué tabla? ¿Qué columna?
- ❓ ¿Qué significa que en la tabla clientes el type sea ALL? ¿Qué implica esto para el rendimiento si hay muchos registros?
- ❓ ¿Qué indica el valor ref en la columna type de la tabla pedidos?
- ❓ Compará los valores en la columna rows. ¿Qué indica la diferencia?
- ❓ ¿Cómo podrías mejorar el rendimiento de esta consulta? (pensá en índices, reescritura, etc.)

#### ¿QUERÉS PONERTE UN POCO MÁS A PRUEBA?

Probá ejecutar la consulta con y sin el comando **EXPLAIN**.

- ❓ ¿Notás diferencias si usás un índice sobre total en la tabla pedidos?

**BASE DE DATOS II****CONTENIDOS Y ACTIVIDADES****ACTIVIDAD INTENSIVA 02****SEGUNDA PRÁCTICA****CREAMOS NUESTRA BASE DE DATOS DE PRUEBA Y LUEGO LE AÑADIMOS DOS TABLAS**

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100)  
);
```

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    cliente_id INT,  
    total DECIMAL(10, 2),  
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

**LE AÑADIMOS ALGUNOS DATOS DE EJEMPLO**

```
INSERT INTO clientes (id, nombre) VALUES  
(1, 'Ana'), (2, 'Bruno'), (3, 'Carla');
```

```
INSERT INTO pedidos (id, cliente_id, total) VALUES  
(1, 1, 120.50), (2, 2, 75.00), (
```

**AHORA EJECUTAREMOS LA CONSULTA Y LA ANALIZAREMOS**

```
EXPLAIN SELECT c.nombre, p.total
```

```
FROM clientes c
```

```
JOIN pedidos p ON c.id = p.cliente_id
```

```
WHERE p.total > 100;
```

## BASE DE DATOS II

## CONTENIDOS Y ACTIVIDADES

## ANÁLISIS DE LA TABLA DE SALIDA

Observá los siguientes campos clave del resultado del **EXPLAIN**:

**id**: orden de ejecución.

**select\_type**: tipo de consulta (simple, subconsulta, etc.).

**table**: tabla que se está accediendo.

**type**: tipo de acceso (ALL, index, ref, eq\_ref, etc.).

**possible\_keys**: claves que podrían usarse.

**key**: clave realmente utilizada.

**ref**: columna que se compara con el índice.

**rows**: estimación de filas que se van a escanear.

**Extra**: pasos adicionales del optimizador (como "Using where").

## PREGUNTAS PARA RESPONDER

- ❓ ¿Qué tipo de acceso (type) se utilizó para cada tabla?
- ❓ ¿Cuántas filas se estimó que se iban a leer en cada tabla (rows)?
- ❓ ¿Se usó algún índice? ¿Cuál (key)?
- ❓ ¿Qué podés deducir del campo Extra?
- ❓ ¿Cómo podrías mejorar el rendimiento de esta consulta?

## ¿LO MEJORAMOS?

Agregar un índice sobre pedidos.total y repetir el análisis:

```
CREATE INDEX idx_total ON pedidos(total);
```

## REPETIR LA CONSULTA EXPLAIN

```
EXPLAIN SELECT c.nombre, p.total
```

```
FROM clientes c
```

```
JOIN pedidos p ON c.id = p.cliente_id
```

```
WHERE p.total > 100;
```